

THE PROCESS OF INTERACTION DESIGN

Introduction

Design is a practical and creative activity with the aim of developing a product that helps its users achieve their goals. **Interaction design** is concerned with 'designing interactive products to support the way people communicate and interact in their everyday and working lives.' But how do you go about doing this? Developing a product must begin with gaining some understanding of what is required of it, but where do these requirements come from? Whom do you ask about them? Underlying good interaction design is the philosophy of user-centered design, i.e. involving users throughout development, but who are the users? Will they know what they want or need even if we can find them to ask? For an innovative product, users are unlikely to be able to envision what is possible, so where do these ideas come from?

What Is Involved in Interaction Design?

There are many fields of design; for example, graphic design, architectural design, industrial and software design, and although each discipline has its own interpretation of 'designing,' there are three fundamental activities that are recognized in all design: understanding the requirements, producing a design that satisfies those requirements, and evaluating the design.

Interaction design also involves these activities, and in addition we focus attention very clearly on users and their goals. For example, we investigate the artifact's use and target domain by taking a user-centered approach to development, we seek users' opinions and reactions to early designs, and we involve users appropriately in the development process itself. This means that users' concerns direct the development rather than just technical concerns, and for interaction design, the three fundamental activities of design listed above are extended to include an activity of prototyping so that users can interact with the design.

So design involves work on requirements, designing a solution, producing an interactive version of the solution, and evaluating it. But design is also about trade-offs, about balancing conflicting requirements. One common form of trade-off when developing a system to offer advice is deciding how much choice will be given to the user and how much direction the system should offer. Often the division will depend on the purpose of the system, e.g. for business travel or for vacations. Getting the balance right requires experience, but it also requires the development and evaluation of alternative solutions.

Generating alternatives is a key principle in most design disciplines, and one that should be encouraged in interaction design. Linus Pauling, twice a Nobel Prize winner, once said: "The best way to get a good idea, is to get lots of ideas." This is not necessarily easy, however, and unlike many design disciplines, interaction designers are not generally trained to generate alternative designs. The good news is that the ability to brainstorm and contribute alternative ideas can be learned. For example, Kelley (2008) describes seven secrets for better brainstorms, including sharpen the focus (have a well-honed problem statement), playful rules (to encourage ideas),

and get physical (use visual props). He also discusses six ways to kill a brainstorm, including do it off-site and write everything down.

Involving users and others in the design process means that the designs and potential solutions will need to be communicated to people other than the original designer. This requires the design to be captured and expressed in some suitable form that allows review, revision, and improvement. There are many ways of doing this, one of the simplest being to produce a series of sketches. Other common approaches are to write a description in natural language, to draw a series of diagrams, and to build prototypes. A combination of these techniques is likely to be the most effective. When users are involved, capturing and expressing a design in a suitable format is especially important since they are unlikely to understand jargon or specialist notations. In fact, a form that users can interact with is most effective, and building prototypes is an extremely powerful approach.

The Value of Prototyping

I learned the value of prototyping through a very effective role-playing exercise. I was on a course designed to introduce new graduates to different possible careers in industry. One of the themes was production and manufacturing and the aim of one group exercise was to produce a book. Each group was told that it had 30 minutes to deliver 10 books to the person in charge. Groups were given various pieces of paper, scissors, sticky tape, staples, etc., and told to organize ourselves as best we could. So in my group we set to work organizing ourselves into a production line, with one of us cutting up the paper, another stapling the pages together, another sealing the binding with the sticky tape, and so on. One person was even in charge of quality assurance. It took us less than 10 minutes to produce the 10 books, and we rushed off with our delivery. When we showed the person in charge, he replied, 'That's not what I wanted; I need it bigger than that.' Of course, the size of the notebook wasn't specified in the description of the task, so we found out how big he wanted it, got some more materials, and scooted back to produce 10 more books. Again, we set up our production line and produced 10 books to the correct size. On delivery, we were again told that it was not what was required: he wanted the binding down the other edge. This time we got as many of the requirements as we could and went back, developed one book, and took that back for further feedback and refinement before producing the 10 required.

If we had used prototyping as a way of exploring our ideas and checking requirements in the first place, we could have saved so much effort and resource! ■

The Importance of Involving Users

The description above emphasizes the need to involve users in interaction design, but why is it important? Before the impact that user involvement can have on project success was recognized, it was common for developers to talk to managers or to proxy users, i.e. people who role-played

as users, when eliciting requirements, or even to use their own judgment without reference to anyone else. While a proxy user can provide useful information, they will not have the same perspective as someone who performs the task every day, or who will use the intended product on a regular basis. For example, several years ago, I was involved with a system to process and record financial transactions from the foreign exchange (forex) dealers in a large international bank. The users of this system took the handwritten transaction records completed by the forex dealers and entered the details into the system. The system then validated the transaction and communicated a confirmation to the relevant parties. When the requirements for this system were developed, no one from the development team spoke to the end-users of the system; the requirements were identified by higher level managers. Although the system was successfully used to support the task, the end-users had developed several work-arounds and crib sheets. For example, each desk had a sheet of paper with lists of buyer codes and names. The system required both buyer code and buyer name to be entered, but it was quite common for the dealers to write only the buyer name on the transaction record. The list of names and codes was used to identify the codes manually.

The best way to ensure that development continues to take users' activities into account is to involve real users throughout development. In this way, developers can gain a better understanding of users' goals, leading to a more appropriate, more usable product. However, two other aspects that have nothing to do with functionality are equally as important if the product is to be usable and used: expectation management and ownership.

Expectation management is the process of making sure that the users' expectations of the new product are realistic. The purpose of expectation management is to ensure that there are no surprises for users when the product arrives. If users feel they have been cheated by promises that have not been fulfilled, then this will cause resistance and even rejection. Marketing of the new arrival must be careful not to misrepresent the product, although it may be particularly difficult to achieve with a large and complex system (Nevo and Wade, 2007). How many times have you seen an advert for something you thought would be really good to have, but when you see one, discover that the marketing hype was a little exaggerated? I expect you felt quite disappointed and let down. This is the kind of feeling that expectation management tries to avoid.

It is better to exceed users' expectations than to fall below them. This does not mean adding more features, but that the product supports the users' goals more effectively than they expect. Involving users throughout development helps with expectation management because they can see from an early stage what the product's capabilities are. They will also understand better how it will affect their jobs and lives, and why the features are designed that way. Adequate and timely training is another technique for managing expectations. If users have the chance to work with the product before it is released, through training or hands-on demonstrations of a prerelease version, then they will understand better what to expect when the final product is available.

A second reason for user involvement is ownership. Users who are involved and feel that they have contributed to a product's development are more likely to feel a sense of ownership towards it and support its use.

Four Approaches to Interaction Design

Saffer (2010) suggests four main approaches to interaction design, each of which is based on a distinct underlying philosophy: user-centered design, activity-centered design, systems design, and genius design. He acknowledges that the purest form of any of these is unlikely to be realized, and takes an extreme view of each in order to distinguish between them. In user-centered design, the user knows best and is the only guide to the designer; the designer's role is to translate the users' needs and goals into a design solution. Activity-centered design focuses on the behavior surrounding particular tasks. Users still play a significant role but it is their behavior rather than their goals and needs that are important. Systems design is a structured, rigorous, and holistic design approach that focuses on context and is particularly appropriate for complex problems. In systems design it is the system (i.e. the people, computers, objects, devices, and so on) that are the center of attention while the users' role is to set the goals of the system.

Finally, genius design is different from the other three approaches because it relies solely on the experience and creative flair of a designer. Jim Leftwich, an experienced interaction designer interviewed in Saffer (2010, pp. 44–5) prefers the term 'rapid expert design.' In this approach the users' role is to validate ideas generated by the designer, and users are not involved during the design process itself. Saffer points out that this is not necessarily by choice, but may be due to limited or no resources for user involvement. Apple, for example, does very little user research or testing, yet the Apple iPod is acknowledged as a significant design achievement.

Different design problems lend themselves more easily to different approaches, and different designers will tend to gravitate towards using the approach that suits them best. Although an individual designer may prefer a particular approach, it is important that the approach for any one design problem is chosen with that design problem in mind. ■

What Is a User-Centered Approach?

Throughout this course, we emphasize the need for a user-centered approach to development. By this we mean that the real users and their goals, not just technology, are the driving force behind product development. As a consequence, a well-designed system will make the most of human skill and judgment, will be directly relevant to the activity in hand, and will support rather than constrain the user. This is less of a technique and more of a philosophy. In 1985, Gould and Lewis (1985) laid down three principles they believed would lead to a 'useful and easy to use computer system':

1. Early focus on users and tasks. This means first understanding who the users will be by directly studying their cognitive, behavioral, anthropomorphic, and attitudinal characteristics. This

requires observing users doing their normal tasks, studying the nature of those tasks, and then involving users in the design process.

2. Empirical measurement. Early in development, the reactions and performance of intended users to printed scenarios, manuals, etc., are observed and measured. Later on, users interact with simulations and prototypes and their performance and reactions are observed, recorded, and analyzed.

3. Iterative design. When problems are found in user testing, they are fixed and then more tests and observations are carried out to see the effects of the fixes. This means that design and development is iterative, with cycles of design-test-measure-redesign being repeated as often as necessary.

These three principles are now accepted as the basis for a user-centered approach (e.g. see Mao et al, 2005) but when Gould and Lewis wrote their paper, they were not accepted by most developers. In fact, they remark in their paper that when they started recommending these to designers, the designers' reactions implied that these principles were obvious. However, when they asked designers at a human factors symposium for the major steps in software design, most of them did not cite many of the principles – in fact, only 2% mentioned all of them. So maybe they had obvious merit, but they were not so easy to put into practice. The Olympic Messaging System (OMS) (Gould et al, 1987) was the first reported large computer-based system to be developed using these three principles. Here a combination of techniques was used to elicit users' reactions to designs, from the earliest prototypes through to the final product. In this case, users were mainly involved in evaluating designs. Below, we discuss these principles in more detail.

Early Focus on Users and Tasks

This principle can be expanded and clarified through the following five further principles:

1. Users' tasks and goals are the driving force behind the development. In a user-centered approach to design, while technology will inform design options and choices, it should not be the driving force. Instead of saying 'Where can we deploy this new technology?' say 'What technologies are available to provide better support for users' goals?'

2. Users' behavior and context of use are studied and the system is designed to support them. This is about more than just capturing the tasks and the users' goals. How people perform their tasks is also significant. Understanding behavior highlights priorities, preferences, and implicit intentions. One argument against studying current behavior is that we are looking to improve work, not to capture bad habits in automation. The implication is that exposing designers to users is likely to stifle innovation and creativity, but experience tells us that the opposite is true (Beyer and Holtzblatt, 1998). In addition, if something is designed to support an activity with little understanding of the real work involved, it is likely to be incompatible with current practice, and

users don't like to deviate from their learned habits if operating a new device with similar properties (Norman, 2013).

3. Users' characteristics are captured and designed for. When things go wrong with technology, we often say that it is our fault. But as humans, we are prone to making errors and we have certain limitations, both cognitive and physical. Products designed to support humans should take these limitations into account and should limit the mistakes we make. Cognitive aspects such as attention, memory, and perception issues were introduced. Physical aspects include height, mobility, and strength. Some characteristics are general, such as about one man in 12 has some form of color blindness, but some characteristics may be associated more with the job or particular task at hand. So, as well as general characteristics, we need to capture those specific to the intended user group.

4. Users are consulted throughout development from earliest phases to the latest and their input is seriously taken into account. As discussed above, there are different levels of user involvement and there are different ways in which to consult users. However, involvement is organized, it is important that users are respected by designers.

5. All design decisions are taken within the context of the users, their work, and their environment. This does not necessarily mean that users are actively involved in design decisions.

Assume that you are involved in developing a novel online experience for buying garden plants. Although several websites exist for buying plants online, you want to produce a distinct experience to increase the organization's market share. Suggest ways of applying the above principles in this task.

Empirical Measurement

Specific usability and user experience goals should be identified, clearly documented, and agreed upon at the beginning of the project. They can help designers to choose between alternative designs and to check on progress as the product is developed. Identifying specific goals up front means that the product can be empirically evaluated at regular stages as it is developed.

Iterative Design

Iteration allows designs to be refined based on feedback. As users and designers engage with the domain and start to discuss requirements, needs, hopes, and aspirations, then different insights into what is needed, what will help, and what is feasible will emerge. This leads to a need for iteration, for the activities to inform each other and to be repeated. However good the designers are and however clear the users may think their vision is of the required artifact, it will be necessary to revise ideas in light of feedback, several times. This is particularly true when trying to innovate. Innovation rarely emerges whole and ready to go. It takes time, evolution, trial and error, and a great deal of patience. Iteration is inevitable because designers never get the solution right the first time.

Four Basic Activities of Interaction Design

Four basic activities for interaction design include; establishing requirements for the user experience, designing alternatives that meet those requirements, prototyping the alternative designs so that they can be communicated and assessed, and evaluating what is being built throughout the process and the user experience it offers. They are fairly generic activities and can be found in other design disciplines too, such as graphic design, architectural design, and product design.

Establishing Requirements

In order to design something to support people, we must know who our target users are and what kind of support an interactive product could usefully provide. These needs form the basis of the product's requirements and underpin subsequent design and development. This activity is fundamental to a user-centered approach, and is very important in interaction design. Understanding these needs is gleaned through data gathering and analysis.

Designing Alternatives

This is the core activity of designing: actually suggesting ideas for meeting the requirements. This activity can be viewed as two sub-activities: conceptual design and concrete design. Conceptual design involves producing the conceptual model for the product, and a conceptual model describes an abstraction outlining what people can do with a product and what concepts are needed to understand how to interact with it. Concrete design considers the detail of the product including the colors, sounds, and images to use, menu design, and icon design. Alternatives are considered at every point.

Prototyping

Interaction design involves designing interactive products. The most sensible way for users to evaluate such designs is to interact with them, and this can be achieved through prototyping. This does not necessarily mean a piece of software is required. There are different prototyping techniques, not all of which require a working piece of software. For example, paper-based prototypes are very quick and cheap to build and are very effective for identifying problems in the early stages of design, and through role-playing users can get a real sense of what it will be like to interact with the product.

Evaluating

Evaluation is the process of determining the usability and acceptability of the product or design that is measured in terms of a variety of usability and user experience criteria. Interaction design requires a high level of user involvement throughout development, and this enhances the chances of an acceptable product being delivered. Evaluation does not replace the activities concerned with quality assurance and testing to make sure that the final product is fit for purpose, but it complements and enhances them.

The activities of establishing requirements, designing alternatives, building prototypes, and evaluating them are intertwined: alternatives are evaluated through the prototypes and the results are fed back into further design or might identify missing requirements. This iteration is one of the key characteristics of a user-centered approach.

A Simple Lifecycle Model for Interaction Design

Understanding what activities are involved in interaction design is the first step to being able to do it, but it is also important to consider how the activities are related to one another so that the full development process can be seen. The term lifecycle model is used to represent a model that captures a set of activities and how they are related. Software engineering has spawned many lifecycle models including the waterfall, spiral, and RAD (rapid applications development) models (Sommerville, 2010). One lifecycle model that was not widely used, but which was intended to capture the iterative nature of software development, was called the fountain lifecycle model (Henderson-Sellers and Edwards, 1993). The field of HCI has also been associated with several lifecycle models such as the Star (Hartson and Hix, 1989) and an international standard model ISO 9241-210.

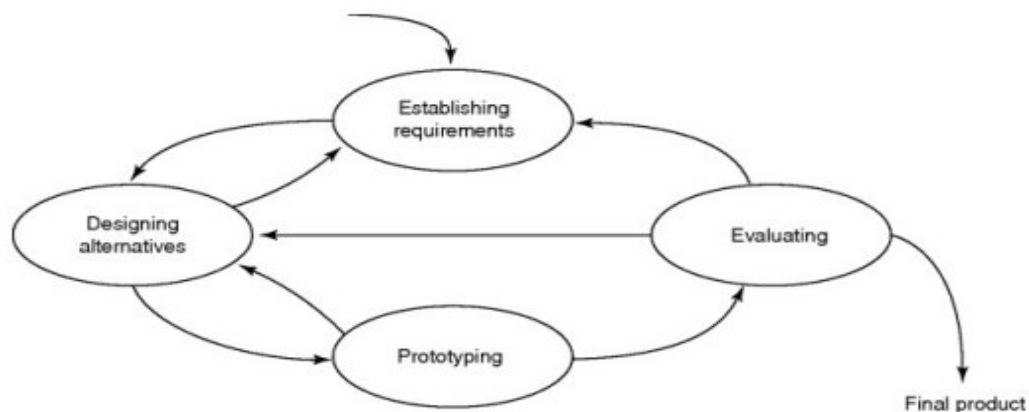
Existing models have varying levels of sophistication and complexity. For projects involving only a few experienced developers, a simple process would probably be adequate. However, for larger systems involving tens or hundreds of developers with hundreds or thousands of users, a simple process just isn't enough to provide the management structure and discipline necessary to engineer a usable product. So something is needed that will provide more formality and more discipline. Note that this does not necessarily mean that innovation is lost or that creativity is stifled, just that a structured process is used to provide a more stable framework for creativity.



However simple or complex it appears, any lifecycle model is a simplified version of reality. It is intended as an abstraction and, as with any good abstraction, only the amount of detail required for the task at hand will be included. Any organization wishing to put a lifecycle model into practice will need to add detail specific to its particular circumstances and culture.

The activities of interaction design are related as shown in the figure below. This model incorporates the four activities of interaction design and the three principles of user-centered design discussed above. Depending on the kind of product being developed, it may not be possible or appropriate to follow this model for every aspect, and more detail would be required to put the lifecycle into practice in a real project. We have not specified outputs from each activity, although measurable usability and user experience criteria would be specified early on and referred to across all the activities.

The model is not intended to be prescriptive; that is, we are not suggesting that this is how all interactive products are or should be developed. It is based on our observations of interaction design and on information we have gleaned in the research for this book. It has its roots in the software engineering and HCI lifecycle models mentioned above and it represents what we believe is practiced in the field.



Most projects start by establishing requirements. The project may have arisen because of some evaluation that has been done, but the lifecycle of the new (or modified) product can be thought of as starting at this point. From this activity, some alternative designs are generated in an attempt to meet the requirements that have been identified. Then prototype versions of the designs are developed and evaluated. Based on the feedback from the evaluations, the team may need to return to identify more needs or refine requirements, or it may go straight into redesigning. It may be that more than one alternative design follows this iterative cycle in parallel with others, or it may be that one alternative at a time is considered. Implicit in this cycle is that the final product will emerge in an evolutionary fashion from a rough initial idea through to the finished product. Exactly how this evolution happens may vary from project to project. The only factor limiting the number of times through the cycle is the resources available, but whatever the number is, development ends with an evaluation activity that ensures the final product meets the prescribed user experience and usability criteria.

Interaction Design and usability/the User Experience

Part of the process of understanding users is to be clear about the primary objective of developing an interactive product for them. Is it to design an efficient system that will allow them

to be highly productive in their work, or is it to design a learning tool that will be challenging and motivating, or is it something else? To help identify the objectives we suggest classifying them in terms of usability and user experience goals. Traditionally, usability goals have been viewed as being concerned with meeting specific usability criteria, e.g. efficiency, whereas, more recently, user experience goals have been concerned with explicating the nature of the user experience, e.g. to be aesthetically pleasing. It is important to note, however, that the distinction between the two types of goal is not clear-cut, since usability is fundamental to the quality of the user experience and, conversely, aspects of the user experience, such as how it feels and looks, are inextricably linked with how usable the product is. We distinguish between them here to help clarify their roles but stress the importance of considering them together when designing for a user experience. Also, historically, HCI was concerned primarily with usability (known as usability engineering) but has since become concerned with understanding, designing for, and evaluating a wider range of user experience aspects.

Usability

Usability and user experience (UX) are not the same thing: the usability of a product is a crucial part that shapes its UX, and hence falls under the umbrella of UX. While many might think that usability is solely about the “ease of use” of a product, it actually involves a great deal more than that. So, let’s find out more about usability here and be absolutely certain about the nature of this fundamental building block of our craft!

The ISO 9241-11 standard on usability describes it as:

“The extent to which a product can be used by specified users to achieve specified goals, with effectiveness, efficiency and satisfaction in a specified context of use.”

– ISO 9241 Ergonomics of Human System Interaction

Usability is hence more than just about whether users can perform tasks easily (ease-of-use). It also deals with user satisfaction—for a website to be usable, it has to be engaging and aesthetically pleasing, too.

Usability refers to ensuring that interactive products are easy to learn, effective to use, and enjoyable from the user's perspective. It involves optimizing the interactions people have with interactive products to enable them to carry out their activities at work, at school, and in their everyday lives

The 5 Characteristics of Usable Products

In 2001, Whitney Quesenbery, the UX and Usability Expert and former President of the Usability Professionals’ Association (UXPA), offered five criteria that a product must meet to be usable:

- Effectiveness
- Efficiency

- Engagingness
- Error Tolerance
- Ease of Learning

1. Effectiveness

Effectiveness is about whether users can complete their goals with a high degree of accuracy. Much of the effectiveness of a product comes from the support provided to users when they work with the product; for example, fixing a credit card field so that it only accepts a valid credit card number entry can reduce data entry errors and help users perform their tasks correctly. There are many different ways to provide support—the key is to be as *informative* as possible in a *meaningful* way to the user.

2. Efficiency

Effectiveness and efficiency have come to be blurred in the mind. They are, however, quite different from a usability perspective. Efficiency is all about speed. How fast can the user get the job done?

You'll want to examine the number of steps (or indeed clicks/keystrokes) to achieving the objective; can they be reduced? This will help develop efficient processes. Clearly labeled navigation buttons with obvious uses will also help, as will the development of meaningful shortcuts (think about the number of hours you've saved using Ctrl+C and Ctrl+V to copy and paste text).

So as to maximize efficiency, you need to examine how your users prefer to work—are they interacting via a smartphone or a desktop computer with a large keyboard and mouse? The two require very different approaches to navigation.

3. Engagement

Engagement refers to the level of engagement a system offers. Indeed, engagement has become something of a buzzword, but if you cut through the fluff, you'll find that engagement occurs when the user finds the product pleasant and gratifying to use. Aesthetics matter here, and it's why many companies invest a small fortune in graphic design elements—but they're not the only factors involved in how engaging a design is.

Engagement is not only about looking nice; it's also about looking *right*. Proper layouts, readable typography and ease of navigation all come together to deliver the right interaction for the user and make it engaging. Looking nice isn't everything, as Wikipedia (famous for its ultra-basic design) proves.

4. Error Tolerance

It seems unlikely that—for any degree of complexity—you can *completely* eliminate errors in products; in particular, digital products may be error-prone because of the ecosystem in which they dwell – an ecosystem which is beyond the designer's control.

However, the next best thing is to minimize errors from occurring and to ensure that a user can easily recover from an error and get back to what he or she was doing. This is error tolerance.

Promoting error tolerance, according to Whitney Quesenberry, requires:

- Restricting opportunities to do the wrong thing. Make links/buttons clear and distinct, keep language clear and simple, don't use jargon unless absolutely necessary and keep dependencies in forms or actions together. Limit options to correct choices if you can and give examples and support when asking people to provide data.

- Offering the opportunity to “redo”. Give users a way to reset what they’ve just done and go back and start again.
- Assume everyone is going to do things you don’t expect them to do. Then, either facilitate that or offer advice/support to get back on the right path.

5. Ease of Learning

If you want a product to be used regularly, then you want the users to be able to learn the product *easily* so that when they use it again, it comes as second nature.

You also need to accommodate ease of learning when releasing new functionality and features; otherwise, a familiar and happy user may quickly become frustrated with your latest release. This is something that tends to happen a lot on social networks; whenever a new set of features is released, they tend to be greeted with howls of outrage from comfortable users, even if they’re better than the old features. And this is true even when the new features are also easy to learn. Facebook’s switch to the ‘Timeline’ format in 2012 (after giving users months to shift from the previous format) offered a prime example of this curious reaction from some users.

The best way to support ease of learning is to design systems that match a user’s existing mental models. A mental model is simply a representation of something in the real world and how it is done from the user’s perspective. It’s why virtual buttons look a lot like real buttons – we know that we push buttons. The form elicits the appropriate action in the user, hence making it easy to learn.

Utility + Usability = Usefulness

When you’re designing for usability, it’s important to think about *utility*, too. While usability is concerned with making functions easy and pleasant to use, utility is about providing functions that users *need* in the first place.

Only when usability is combined with utility do products become useful to their users. A mobile payment app could provide the most usable feature of adding the people around you on Facebook, but because most users of that app wouldn’t need that feature, it’s going to be useless to them. All your effort towards building the most user-friendly feature could go to waste if that feature isn’t needed.

Other Usability Goals

More specifically, usability is broken down into the following goals:

- effective to use (effectiveness)
- efficient to use (efficiency)
- safe to use (safety)
- having good utility (utility)
- easy to learn (learnability)
- easy to remember how to use (memorability).

Usability goals are typically operationalized as questions. The purpose is to provide the interaction designer with a concrete means of assessing various aspects of an interactive product and the user experience. Through answering the questions, designers can be alerted very early on in the design process to potential design problems and conflicts that they might not have considered. However, simply asking ‘is the system easy to learn?’ is not going to be very helpful.

Asking about the usability of a product in a more detailed way – for example, ‘how long will it take a user to figure out how to use the most basic functions for a new smartwatch; how much can they capitalize on from their prior experience; and how long would it take a user to learn the whole set of functions?’ – will elicit far more information. Below we give a description of each goal and a question for each one.

- Effectiveness is a very general goal and refers to how good a product is at doing what it is supposed to do.

Question: Is the product capable of allowing people to learn, carry out their work efficiently, access the information they need, or buy the goods they want?

- Efficiency refers to the way a product supports users in carrying out their tasks. The marble answering machine described at the beginning of this chapter was considered efficient in that it let the user carry out common tasks, e.g. listening to messages, through a minimal number of steps. In contrast, the voice mail system was considered inefficient because it required the user to carry out many steps and learn an arbitrary set of sequences for the same common task. This implies that an efficient way of supporting common tasks is to let the user use single button or key presses. An example of where this kind of efficiency mechanism has been employed effectively is in online shopping. Once users have entered all the necessary personal details in an online form to make a purchase, they can let the website save all their personal details. Then, if they want to make another purchase at that site, they don't have to re-enter all their personal details again. A highly successful mechanism patented by Amazon.com is the one-click option, which requires users only to click a single button when they want to make another purchase.

Question: Once users have learned how to use a product to carry out their tasks, can they sustain a high level of productivity?

- Safety involves protecting the user from dangerous conditions and undesirable situations. In relation to the first ergonomic aspect, it refers to the external conditions where people work. For example, where there are hazardous conditions – such as X-ray machines or toxic chemicals – operators should be able to interact with and control computer-based systems remotely. The second aspect refers to helping any kind of user in any kind of situation avoid the dangers of carrying out unwanted actions accidentally. It also refers to the perceived fears users might have of the consequences of making errors and how this affects their behavior. To make interactive products safer in this sense involves (i) preventing the user from making serious errors by reducing the risk of wrong keys/buttons being mistakenly activated (an example is not placing the quit or delete-file command right next to the save command on a menu) and (ii) providing users with various means of recovery should they make errors. Safe interactive systems should engender confidence and allow the user the opportunity to explore the interface to try out new operations (see Figure 1.8a). Other safety mechanisms include undo facilities and

confirmatory dialog boxes that give users another chance to consider their intentions (a well-known example is the appearance of a dialog box, after issuing the command to delete everything in the trashcan, saying: 'Are you sure you want to remove all the items in the Trash permanently?' – see Figure 1.8b).



Figure 1.8 (a) A safe and unsafe menu. Which is which and why? (b) A warning dialog box for Mac OS X

Question: What is the range of errors that are possible using the product and what measures are there to permit users to recover easily from them?

- Utility refers to the extent to which the product provides the right kind of functionality so that users can do what they need or want to do. An example of a product with high utility is an accounting software package that provides a powerful computational tool that accountants can use to work out tax returns. An example of a product with low utility is a software drawing tool that does not allow users to draw freehand but forces them to use a mouse to create their drawings, using only polygon shapes.

Question: Does the product provide an appropriate set of functions that will enable users to carry out all their tasks in the way they want to do them?

- Learnability refers to how easy a system is to learn to use. It is well known that people don't like spending a long time learning how to use a system. They want to get started straight away and become competent at carrying out tasks without too much effort. This is especially so for interactive products intended for everyday use (e.g. social media, email, GPS) and those used only infrequently (e.g. online tax forms). To a certain extent, people are prepared to spend longer learning more complex systems that provide a wider range of functionality, like web authoring tools. In these situations, pop-up tutorials can

help by providing contextualized step-by-step material with hands-on exercises. A key concern is determining how much time users are prepared to spend learning a product. It seems a waste if a product provides a range of functionality that the majority of users are unable or not prepared to spend time learning how to use.

Question: Is it possible for the user to work out how to use the product by exploring the interface and trying out certain actions? How hard will it be to learn the whole set of functions in this way?

- Memorability refers to how easy a product is to remember how to use, once learned. This is especially important for interactive products that are used infrequently. If users haven't used an operation for a few months or longer, they should be able to remember or at least rapidly be reminded how to use it. Users shouldn't have to keep relearning how to carry out tasks. Unfortunately, this tends to happen when the operations required to be learned are obscure, illogical, or poorly sequenced. Users need to be helped to remember how to do tasks. There are many ways of designing the interaction to support this. For example, users can be helped to remember the sequence of operations at different stages of a task through meaningful icons, command names, and menu options. Also, structuring options and icons so they are placed in relevant categories of options, e.g. placing all the drawing tools in the same place on the screen, can help the user remember where to look to find a particular tool at a given stage of a task.

Question: What kinds of interface support have been provided to help users remember how to carry out tasks, especially for products and operations they use infrequently?

As well as couching usability goals in terms of specific questions, they are turned into usability criteria. These are specific objectives that enable the usability of a product to be assessed in terms of how it can improve (or not) a user's performance. Examples of commonly used usability criteria are time to complete a task (efficiency), time to learn a task (learnability), and the number of errors made when carrying out a given task over time (memorability). These can provide quantitative indicators of the extent to which productivity has increased, or how work, training, or learning have been improved. They are also useful for measuring the extent to which personal, public, and home-based products support leisure and information gathering activities. However, they do not address the overall quality of the user experience, which is where user experience goals come into play.

User Experience Goals

The user experience (UX) is central to interaction design. By this it is meant how a product behaves and is used by people in the real world. Nielsen and Norman (2014) define it as encompassing “all aspects of the end-user's interaction with the company, its services, and its products.” As stressed by Garrett (2010, p. 10), “every product that is used by someone has a user experience: newspapers, ketchup bottles, reclining armchairs, cardigan sweaters.” More specifically, it is about how people feel about a product and their pleasure and satisfaction when using it, looking at it, holding it, and opening or closing it. It includes their overall impression of

how good it is to use, right down to the sensual effect small details have on them, such as how smoothly a switch rotates or the sound of a click and the touch of a button when pressing it. An important aspect is the quality of the experience someone has, be it a quick one, such as topping up a cell phone, a leisurely one, such as playing with an interactive toy, or an integrated one, such as visiting a museum (Law et al, 2009). It is important to point out that one cannot design a user experience, only design for a user experience. In particular, one cannot design a sensual experience, but only create the design features that can evoke it. For example, the outside case of a cell phone can be designed to be smooth silky, and fit in the palm of a hand; when held, touched, looked at, and interacted with, that can provoke a sensual and satisfying user experience. Conversely, if it is designed to be heavy and awkward to hold, it is much more likely to end up providing a poor user experience, one that is uncomfortable and unpleasant.

Designers sometimes refer to UX as UXD. The addition of the D to UX is meant to encourage design thinking that focuses on the quality of the user experience rather than on the set of design methods to use (Allanwood and Beare, 2014). As Norman (2004) has stressed for many years, “It is not enough that we build products that function, that are understandable and usable, we also need to build joy and excitement, pleasure and fun, and yes, beauty to people's lives.”

There are many aspects of the user experience that can be considered and ways of taking them into account when designing interactive products. Of central importance are the usability, the functionality, the aesthetics, the content, the look and feel, and the sensual and emotional appeal. In addition, Carroll (2004) stresses other wide-reaching aspects, including fun, health, social capital (the social resources that develop and are maintained through social networks, shared values, goals, and norms), and cultural identity, e.g. age, ethnicity, race, disability, family status, occupation, education. At a more subjective level, McCarthy and Wright (2004) discuss the importance of people's expectations and the way they make sense of their experiences when using technology.

How realistic is it for interaction designers to take all of these factors (and potentially many others) into account and, moreover, be able to translate and combine them to produce quality user experiences? Put frankly, there is no magic formula to help them. As of yet, there isn't a unifying theory or framework that can be readily applied by interaction designers. However, there are numerous conceptual frameworks, tried and tested design methods, guidelines, and many relevant research findings – these are described throughout the book. Here, we begin by outlining the process and goals of interaction design.

More generally, McCarthy and Wright's (2004) Technology as Experience framework accounts for the user experience largely in terms of how it is felt by the user. They recognize that defining experience is incredibly difficult because it is so nebulous and ever-present to us, just as swimming in water is to a fish. Nevertheless, they have tried to capture the essence of human experience by describing it in both holistic and metaphorical terms. These comprise a balance of sensual, cerebral, and emotional threads. Their framework draws heavily from the philosophical writings of Dewey and Pragmatism, which focus on the sense-making aspects of human

experience. As Dewey (1934) points out: “Emotion is the moving and cementing force. It selects what is congruous and dyes what is selected with its color, thereby giving qualitative unity to materials externally disparate and dissimilar. It thus provides unity in and through the varied parts of experience.”

McCarthy and Wright propose four core threads that make up our holistic experiences: sensual, emotional, compositional, and spatio-temporal:

- The sensual thread. This is concerned with our sensory engagement with a situation and is similar to the visceral level of Norman's model. It can be equated with the level of absorption people have with various technological devices and applications, most notable being computer games, smartphones, and chat rooms, where users can be highly absorbed in their interactions at a sensory level. These can involve thrill, fear, pain, and comfort.
- The emotional thread. Common examples of emotions that spring to mind are sorrow, anger, joy, and happiness. In addition, the framework points out how emotions are intertwined with the situation in which they arise – e.g. a person becomes angry with a computer because it does not work properly. Emotions also involve making judgments of value. For example, when purchasing a new cell phone, people may be drawn to the ones that are most cool-looking but be in an emotional turmoil because they are the most expensive. They can't really afford them but they really would like one of them.
- The compositional thread. This is concerned with the narrative part of an experience, as it unfolds, and the way a person makes sense of it. For example, when shopping online, the options laid out to people can lead them in a coherent way to making a desired purchase or they can lead to frustrating experiences resulting in no purchase being made. When in this situation, people ask themselves questions such as: What is this about? Where am I? What has happened? What is going to happen next? What would happen if . . . ? The compositional thread is the internal thinking we do during our experiences.
- The spatio-temporal thread. This refers to the space and time in which our experiences take place and their effect upon those experiences. There are many ways of thinking about space and time and their relationship with one another: for example, we talk of time speeding up, standing still, and slowing down, while we talk of space in terms of public and personal places, and needing one's own space.

The threads are meant as ideas to help designers think and talk more clearly and concretely about the relationship between technology and experience. By describing an experience in terms of its interconnected aspects, the framework can aid thinking about the whole experience of a technology rather than as fragmented aspects, e.g. its usability, its marketability, or its utility. For example, when buying clothes online, the framework can be used to capture the whole gamut of experiences, including: the fear or joy of needing to buy a new outfit; the time and place where it can be purchased, e.g. online stores or shopping mall; the tensions of how to engage with the vendor, e.g. the pushy sales assistant or an anonymous website; the value judgment

involved in contemplating the cost and how much one is prepared to spend; the internal monologue that goes on where questions are asked such as will it look good on me, what size should I buy, do I have shoes to match, do I need to try it on, how easy will it be to wash, will I need to iron it each time, and how often will I be able to wear it? All of these aspects can be described in terms of the four threads and in so doing highlight which aspects are more important for a given product. For example, if you were to do this exercise when buying a new car versus a domestic energy-saving device, you would find you would get quite different descriptions.

A diversity of user experience goals has been articulated in interaction design, which cover a range of emotions and felt experiences. These include desirable and undesirable ones, as shown in Table 1.1.

TABLE 1.1**Desirable and undesirable aspects of the user experience**

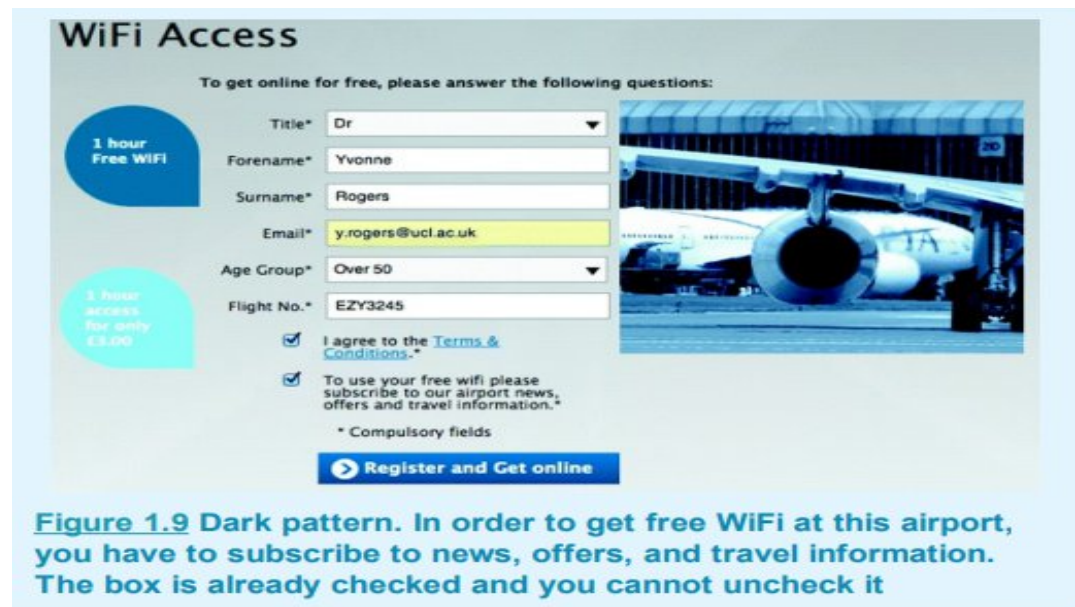
Desirable aspects		
Satisfying	Helpful	Fun
Enjoyable	Motivating	Provocative
Engaging	Challenging	Surprising
Pleasurable	Enhancing sociability	Rewarding
Exciting	Supporting creativity	Emotionally fulfilling
Entertaining	Cognitively stimulating	
Undesirable aspects		
Boring	Unpleasant	
Frustrating	Patronizing	
Making one feel guilty	Making one feel stupid	
Annoying	Cutesy	
Childish	Gimmicky	

Many of these are subjective qualities and are concerned with how a system feels to a user. They differ from the more objective usability goals in that they are concerned with how users experience an interactive product from their perspective, rather than assessing how useful or productive a system is from its own perspective. Whereas the terms used to describe usability goals comprise a small distinct set, many more terms are used to describe the multifaceted nature of the user experience. They also overlap with what they are referring to. In so doing, they offer subtly different options for expressing the way an experience varies for the same activity over time, technology, and place. For example, we may describe listening to music in the shower as highly pleasurable, but consider it more apt to describe listening to music in the car as enjoyable. Similarly, listening to music on a high-end powerful music system may invoke exciting and emotionally fulfilling feelings, while listening to it on an iPod Shuffle may be serendipitously enjoyable, especially not knowing what tune is next. The process of selecting terms that best convey a user's feelings, state of being, emotions, sensations, and so forth when using or interacting with a product at a given time and place can help designers understand the multifaceted and changing nature of the user experience.

Beyond usability: designing to persuade

Schaffer (2009) argues that we should be focusing more on the user experience and less on usability. He points out how many websites are designed to persuade or influence rather than enable users to perform their tasks in an efficient manner. For example, many online shopping sites are in the business of selling services and products, where a core strategy is to entice people to buy what they might not have thought they needed. Online shopping experiences are increasingly about persuading people to buy rather than being designed to make shopping easy. This involves designing for persuasion, emotion, and trust – which may or may not be compatible with usability goals.

This entails determining what customers will do, whether it is to make a donation, buy a product, or renew a membership and involves encouraging, suggesting, or reminding the user of things they might like or need. Many online travel sites try to lure visitors to purchase additional items (such as hotels, insurance, car rental, car parking, day trips) besides the flight they went to book originally and will add a list full of tempting graphics to the visitor's booking form, which they have to scroll through before being able to complete their transaction. These persuasion opportunities need to be designed to be eye-catching and enjoyable – in the same way an array of products are attractively laid out in the aisles of a grocery store that one is required to walk past before reaching one's desired product. Some online sites, however, have gone too far; for example, adding items to the customer's shopping basket (e.g. insurance, special delivery) that the shopper has to deselect if not wanted. This sneaky add-on approach can often result in a negative experience. Shoppers often become annoyed if they notice decisions, that cost money, have been made on their behalf without them even being asked (see Figure 1.9). The key is to nudge people in subtle and pleasant ways that they can trust and feel comfortable with.



Persuasive Technologies and Behavioral Change

A diversity of technologies is increasingly being used to draw people's attention to certain kinds of information in an attempt to change what they do or think. Pop-up ads, warning messages, reminders, prompts, personalized messages, and recommendations are some of the methods that are being deployed on computer screens. Fogg (2003) has labeled this phenomenon persuasive technology; interactive computing systems are deliberately designed to change people's attitudes and behaviors. Traditionally, media such as magazines, newspapers, pamphlets, radio, and TV have been used to persuade people to join a good cause, give up a bad habit, donate money, or buy a product. For example, a picture of a starving child with big round eyes staring out at the reader on the front of a newspaper is commonly used by charities. The effect is to pull at the readers' heartstrings, inducing feelings of guilt and, in so doing, spur them on to sending money.

More recently, interactive techniques have been used to entice, cajole, and persuade people to do something they might not have otherwise done. Successful examples include Amazon's one-click mechanism that makes it so easy and tempting to buy something at their online store, and recommender systems that suggest specific books, hotels, restaurants, etc a reader might want to try based on their previous purchases, choices, and taste. Splash pages to online shopping sites and color images of gorgeous-looking beach and mountain scenes on travel sites are designed to lure people into making impulse purchases.

In addition to using interactive technologies as a more targeted and personalized form of advertising, they can be used to change people's behaviors in non-commercial domains, such as safety, preventative healthcare, fitness, personal relationships, energy consumption, and learning. Here, the emphasis is on changing habits or doing something that will improve an individual's well-being through monitoring their behavior. An early example was Nintendo's Pokémon Pikachu with pedometer attached that was designed to motivate children into being more physically active on a consistent basis. The owner of the digital pet that lives in the device was required to walk, run, or jump each day to keep it alive. The wearer received credits with each step taken – the currency being watts that could be used to buy Pikachu presents. Twenty steps on the pedometer rewarded the player with one watt. If the owner did not exercise for a week, the virtual pet became angry and refused to play anymore. This use of positive rewarding and sulking can be a powerful means of persuasion, given that children often become emotionally attached to their virtual pets, especially when they start to care for them.

Accessibility

Accessibility refers to the degree to which an interactive product is accessible by as many people as possible. A focus is on people with disabilities. But what does it mean to be disabled? Definitions vary, but the following captures the main points. People are considered to be disabled if:

- They have a mental or physical impairment.
- The impairment has an adverse effect on their ability to carry out normal day-to-day activities.
- The adverse effect is substantial and long term (meaning it has lasted for 12 months, or is likely to last for more than 12 months or for the rest of their life).

Whether or not a person is considered to be disabled changes over time with age, or as recovery from an accident progresses. In addition, the severity and impact of an impairment can vary over the course of a day or in different environmental conditions.

It is quite common, when people first consider the topic of accessibility and interaction design, to consider it largely in terms of a specific physical disability, such as the inability to walk or being visually impaired. However, it can often be the case that a person will have more than one disability. There is a wide range of disabilities including:

- Color-blindness: The inability to distinguish between two colors affects approximately 1 in 10 men and 1 in 200 women. This has an impact on the use of color for highlighting or distinguishing interface elements.
- Dyslexia: Although usually associated with difficulties in reading and writing, there are many different forms of dyslexia, some of which affect the way in which people comprehend the totality of concepts. A relatively simple interaction design decision that can cause difficulties for people with dyslexia is the contrast between foreground and background text or images.
- Physical impairments: These range from conditions such as tremor or shaking, weakness, pain, reduced control of limbs, inability to sit upright, to short or missing limbs.

Quesenbery (2009) comments on how accessibility is often considered as making sure there aren't any barriers to access for assistive technologies but without regard to usability, while usability usually targets everyone who uses a site or product, without considering people who have disabilities. The challenge is to create a good user experience for people with disabilities that is both accessible and usable.

Interaction Design Principles

Design principles are used by interaction designers to aid their thinking when designing for the user experience. These are generalizable abstractions intended to orient designers towards thinking about different aspects of their designs. A well-known example is feedback: products should be designed to provide adequate feedback to the users to ensure they know what to do next in their tasks. Another one that has become increasingly important is findability (Morville, 2005). This refers to the degree to which a particular object is easy to discover or locate – be it navigating a website, moving through a building, or finding the delete image option on a digital camera.

Design principles are derived from a mix of theory-based knowledge, experience, and common sense. They tend to be written in a prescriptive manner, suggesting to designers what to provide and what to avoid at the interface – if you like, the dos and don'ts of interaction design. More specifically, they are intended to help designers explain and improve their designs (Thimbleby, 1990). However, they are not intended to specify how to design an actual interface, e.g. telling the designer how to design a particular icon or how to structure a web portal, but act more like triggers to designers, ensuring that they have provided certain features at an interface.

A number of design principles have been promoted. The best known are concerned with how to determine what users should see and do when carrying out their tasks using an interactive product. Here we briefly describe the most common ones: visibility, feedback, constraints, consistency, and affordance.

Visibility.

The importance of visibility is exemplified by our contrasting examples at the beginning of the chapter. The voice mail system made the presence and number of waiting messages invisible, while the answer machine made both aspects highly visible. The more visible functions are, the more likely it is that users will be able to know what to do next. Norman (1988) describes the controls of a car to emphasize this point. The controls for different operations are clearly visible, e.g. indicators, headlights, horn, hazard warning lights, indicating what can be done. The relationship between the way the controls have been positioned in the car and what they do makes it easy for the driver to find the appropriate control for the task at hand.

In contrast, when functions are out of sight, it makes them more difficult to find and know how to use. For example, devices and environments that have become automated through the use of sensor technology (usually for hygiene and energy-saving reasons) – like faucets, elevators, and lights – can sometimes be more difficult for people to know how to control, especially how to activate or deactivate them. This can result in people getting caught out and frustrated (see Figure 1.10). Highly visible controlling devices, like knobs, buttons, and switches, which are intuitive to use, have been replaced by invisible and ambiguous activating zones where people have to guess where to move their hands, bodies, or feet on, into, or in front of to make them work.



Figure 1.10 A sign in the restrooms at Cincinnati airport. Because it is not visible to the user as to what to do to turn the faucet (tap) on and off, a sign has been added to explain what is normally an everyday and well-learned activity. It does not explain, however, what to do if you are wearing black clothing

Feedback.

Related to the concept of visibility is feedback. This is best illustrated by an analogy to what everyday life would be like without it. Imagine trying to play a guitar, slice bread using a knife, or write using a pen if none of the actions produced any effect for several seconds. There would be an unbearable delay before the music was produced, the bread was cut, or the words appeared on the paper, making it almost impossible for the person to continue with the next strum, cut, or stroke.

Feedback involves sending back information about what action has been done and what has been accomplished, allowing the person to continue with the activity. Various kinds of feedback are available for interaction design – audio, tactile, verbal, visual, and combinations of these. Deciding which combinations are appropriate for different kinds of activities and interactivities is central. Using feedback in the right way can also provide the necessary visibility for user interaction.

Constraints.

The design concept of constraining refers to determining ways of restricting the kinds of user interaction that can take place at a given moment. There are various ways this can be achieved. A common design practice in graphical user interfaces is to deactivate certain menu options by shading them gray, thereby restricting the user only to actions permissible at that stage of the activity (see Figure 1.11). One of the advantages of this form of constraining is that it prevents the user from selecting incorrect options and thereby reduces the chance of making a mistake. The use of different kinds of graphical representations can also constrain a person's

interpretation of a problem or information space. For example, flow chart diagrams show which objects are related to which, thereby constraining the way the information can be perceived. The physical design of a device can also constrain how it is used; for example, the external slots in a computer have been designed to only allow a cable or card to be inserted in a certain way. Sometimes, however, the physical constraint is ambiguous, as shown in Figure 1.12.

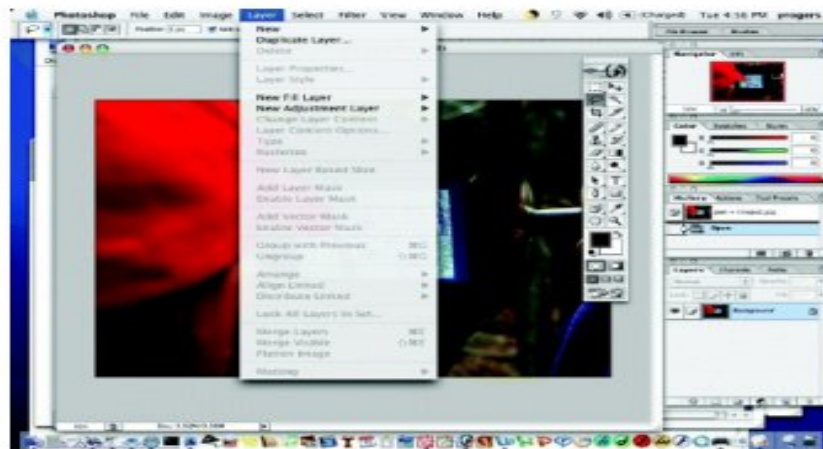


Figure 1.11 A menu showing restricted availability of options as an example of logical constraining. Shaded areas indicate deactivated options

Source: Adobe product box shot reprinted with permission from Adobe Systems Incorporated.

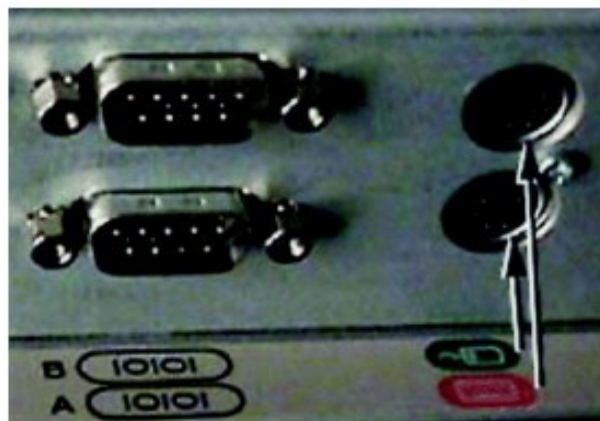


Figure 1.12 Where do you plug in the mouse and keyboard? This figure shows part of the back of a computer. There are two sets of connectors; the two on the right are for a mouse and a keyboard. They look identical and are physically constrained in the same way. How do you know which is which? Do the labels help?

Source: Photograph courtesy of Baddesigns.com.

Consistency.

This refers to designing interfaces to have similar operations and use similar elements for achieving similar tasks. In particular, a consistent interface is one that follows rules, such as using

the same operation to select all objects. For example, a consistent operation is using the same input action to highlight any graphical object at the interface, such as always clicking the left mouse button. Inconsistent interfaces, on the other hand, allow exceptions to a rule. An example is where certain graphical objects (e.g. email messages presented in a table) can be highlighted only by using the right mouse button, while all other operations are highlighted using the left button. A problem with this kind of inconsistency is that it is quite arbitrary, making it difficult for users to remember and making the users more prone to mistakes.

One of the benefits of consistent interfaces, therefore, is that they are easier to learn and use. Users have to learn only a single mode of operation that is applicable to all objects. This principle works well for simple interfaces with limited operations, such as a portable radio with a small number of operations mapped onto separate buttons. Here, all the user has to do is learn what each button represents and select accordingly. However, it can be more problematic to apply the concept of consistency to more complex interfaces, especially when many different operations need to be designed for. For example, consider how to design an interface for an application that offers hundreds of operations, e.g. a word-processing application. There is simply not enough space for a thousand buttons, each of which maps onto an individual operation. Even if there were, it would be extremely difficult and time-consuming for the user to search through them all to find the desired operation. A much more effective design solution is to create categories of commands that can be mapped into subsets of operations.

Affordance.

This is a term used to refer to an attribute of an object that allows people to know how to use it. For example, a mouse button invites pushing (in so doing activating clicking) by the way it is physically constrained in its plastic shell. At a simple level, to afford means 'to give a clue' (Norman, 1988). When the affordances of a physical object are perceptually obvious, it is easy to know how to interact with it. For example, a door handle affords pulling, a cup handle affords grasping, and a mouse button affords pushing. The term has since been much popularized in interaction design, being used to describe how interfaces should make it obvious as to what can be done at them. For example, graphical elements like buttons, icons, links, and scrollbars are talked about with respect to how to make it appear obvious how they should be used: icons should be designed to afford clicking, scrollbars to afford moving up and down, buttons to afford pushing.

Norman (1999) suggests that there are two kinds of affordance: perceived and real. Physical objects are said to have real affordances, like grasping, that are perceptually obvious and do not have to be learned. In contrast, user interfaces that are screen-based are virtual and do not have these kinds of real affordances. Using this distinction, he argues that it does not make sense to try to design for real affordances at the interface – except when designing physical devices, like control consoles, where affordances like pulling and pressing are helpful in guiding the user to know what to do. Alternatively, screen-based interfaces are better conceptualized as perceived affordances, which are essentially learned conventions.

Evaluation

Introduction

The evaluation methods described so far have involved interaction with, or direct observation of, users. Here, we introduce methods that are based on understanding users through knowledge codified in heuristics, or data collected remotely, or models that predict users' performance. None of these methods requires users to be present during the evaluation. Inspection methods typically involve an expert role-playing the users for whom the product is designed, analyzing aspects of an interface, and identifying any potential usability problems by using a set of guidelines. The most well-known are heuristic evaluation and walkthroughs. Analytics involves user interaction logging, which is usually done remotely. Predictive models involve analyzing the various physical and mental operations that are needed to perform particular tasks at the interface and operationalizing them as quantitative measures. One of the most commonly used predictive models is Fitts' Law.

Inspections: Heuristic Evaluation and Walkthroughs

Sometimes users are not easily accessible, or involving them is too expensive or it takes too long. In such circumstances other people, usually referred to as experts, can provide feedback. These are people who are knowledgeable about both interaction design and the needs and typical behavior of users. Various inspection methods were developed as alternatives to usability testing in the early 1990s, drawing on software engineering practice where code and other types of inspections are commonly used. Inspection methods for interaction design include heuristic evaluations, and walkthroughs, in which experts examine the interface of an interactive product, often role-playing typical users, and suggest problems users would likely have when interacting with it. One of the attractions of these methods is that they can be used at any stage of a design project. They can also be used to complement user testing.

Heuristic Evaluation

Heuristic evaluation is a usability inspection method that was developed by Nielsen and his colleagues (Nielsen and Mohlich, 1990; Nielsen, 1994a) and others (Hollingshead and Novick, 2007), and later modified by other researchers for evaluating specific types of systems (e.g. Mankoff et al, 2003; Pinelle et al, 2009). In heuristic evaluation, experts, guided by a set of usability principles known as heuristics, evaluate whether user-interface elements, such as dialog boxes, menus, navigation structure, online help, and so on, conform to tried and tested principles. These heuristics closely resemble high-level design principles (e.g. making designs consistent, reducing memory load, and using terms that users understand). The original set of heuristics for HCI evaluation was developed by Jakob Nielsen and his colleagues who derived them empirically from an analysis of 249 usability problems (Nielsen, 1994b); a revised version of these heuristics is listed below (Nielsen, 2014: useit.com):

Visibility of system status The system should always keep users informed about what is going on, through appropriate feedback within reasonable time.

Match between system and the real world The system should speak the users' language, with words, phrases, and concepts familiar to the user, rather than system-oriented terms. Follow real-world conventions, making information appear in a natural and logical order.

User control and freedom Users often choose system functions by mistake and will need a clearly marked emergency exit to leave the unwanted state without having to go through an extended dialog. Support undo and redo.

Consistency and standards Users should not have to wonder whether different words, situations, or actions mean the same thing. Follow platform conventions.

Error prevention Even better than good error messages is a careful design that prevents a problem from occurring in the first place. Either eliminate error-prone conditions or check for them and present users with a confirmation option before they commit to the action.

Recognition rather than recall Minimize the user's memory load by making objects, actions, and options visible. The user should not have to remember information from one part of the dialog to another. Instructions for use of the system should be visible or easily retrievable whenever appropriate.

Flexibility and efficiency of use Accelerators – unseen by the novice user – may often speed up the interaction for the expert user such that the system can cater to both inexperienced and experienced users. Allow users to tailor frequent actions.

Aesthetic and minimalist design Dialogs should not contain information that is irrelevant or rarely needed. Every extra unit of information in a dialog competes with the relevant units of information and diminishes their relative visibility. Help users recognize, diagnose, and recover from errors Error messages should be expressed in plain language (no codes), precisely indicate the problem, and constructively suggest a solution.

Help and documentation Even though it is better if the system can be used without documentation, it may be necessary to provide help and documentation. Any such information should be easy to search, focused on the user's task, list concrete steps to be carried out, and not be too large.

These heuristics are intended to be used by judging aspects of the interface against them. For example, if a new social networking system is being evaluated, the evaluator might consider how a user would find out how to add friends to her network. The evaluator is meant to go through the interface several times, inspecting the various interaction elements and comparing them with the list of usability principles, i.e. the heuristics. At each iteration, usability problems will be identified or their diagnosis will be refined, until she is satisfied that the majority of them are clear.

Although many heuristics apply to most products (e.g. be consistent and provide meaningful feedback), some of the core heuristics are too general for evaluating products that have come onto the market since Nielsen and Mohlich first developed the method, such as smartphones and other mobile devices, digital toys, online communities, ambient devices, and new web services. Nielsen suggests developing category-specific heuristics that apply to a specific class of product as a supplement to the general heuristics. Evaluators and researchers have therefore typically developed their own heuristics by tailoring Nielsen's heuristics with other design guidelines, market research, and requirements documents. Exactly which heuristics are appropriate and how many are needed for different products is debatable and depends on the goals of the evaluation, but most sets of heuristics have between five and ten items. This number provides a good range of usability criteria by which to judge the various aspects of an interface. More than ten becomes difficult for evaluators to remember; fewer than five tends not to be sufficiently discriminating.

A key question that is frequently asked is how many evaluators are needed to carry out a thorough heuristic evaluation. While one evaluator can identify a large number of problems, she may not catch all of them. She may also have a tendency to concentrate more on one aspect at the expense of missing others. For example, in a study of heuristic evaluation where 19 evaluators were asked to find 16 usability problems in a voice response system allowing customers access to their bank accounts, Nielsen (1992) found a substantial difference between the number and type of usability problems found by the different evaluators. He also notes that while some usability problems are very easy to find by all evaluators, there are some problems that are found by very few experts. Therefore, he argues that it is important to involve multiple evaluators in any heuristic evaluation and recommends between three and five evaluators. His findings suggest that they can typically identify around 75% of the total usability problems.

However, employing multiple experts can be costly. Skillful experts can capture many of the usability problems by themselves and some consultancies now use this technique as the basis for critiquing interactive products – a process that has become known as an expert critique or expert crit in some countries. But using only one or two experts to conduct a heuristic evaluation can be problematic since research has challenged Nielsen's findings and questioned whether even three to five evaluators is adequate. For example, Cockton and Woolrych (2001) and Woolrych and Cockton (2001) point out that the number of experts needed to find 75% of problems depends on the nature of the problems. Their analysis of problem frequency and severity suggests that highly misleading findings can result.

The conclusion from this is that more is better, but more is also more expensive. However, because users and special facilities are not needed for heuristic evaluation and it is comparatively inexpensive and quick, it is popular with developers and is often known as discount evaluation. For a quick evaluation of an early design, one or two experts can probably identify most potential usability problems, but if a thorough evaluation of a fully working prototype is needed then

having a team of experts conducting the evaluation and comparing their findings would be advisable.

Heuristic Evaluation for Websites

In recent years, considerable attention has been focused on the web, mobile apps, and other digital technologies. Heuristics for evaluating websites have become increasingly important and several slightly different sets of heuristics have been developed. Box below contains an extract from a version compiled by web developer Andy Budd that places a stronger emphasis on information content than Nielsen's original heuristics.

Extract from the Heuristics Developed by Budd (2007) that Emphasize Web Design Issues

Clarity

- Make the system as clear, concise, and meaningful as possible for the intended audience.
- Write clear, concise copy
- Only use technical language for a technical audience
- Write clear and meaningful labels
- Use meaningful icons.

Minimize unnecessary complexity and cognitive load

- Make the system as simple as possible for users to accomplish their tasks.
- Remove unnecessary functionality, process steps, and visual clutter
- Use progressive disclosure to hide advanced features
- Break down complicated processes into multiple steps
- Prioritize using size, shape, color, alignment, and proximity.

Provide users with context

Interfaces should provide users with a sense of context in time and space.

- Provide a clear site name and purpose
- Highlight the current section in the navigation
- Provide a breadcrumb trail
- Use appropriate feedback messages
- Show number of steps in a process
- Reduce perception of latency by providing visual cues (e.g. progress indicator) or by allowing users to complete other tasks while waiting.

Promote a pleasurable and positive user experience

The user should be treated with respect and the design should be aesthetically pleasing and promote a pleasurable and rewarding experience.

- Create a pleasurable and attractive design
- Provide easily attainable goals
- Provide rewards for usage and progression

Other heuristics include Shneiderman's Eight Golden Rules of Interface Design. Shneiderman's eight golden rules provide a convenient and succinct summary of the key principles of interface design. They are intended to be used during design but can also be applied, like Nielsen's heuristics, to the evaluation of systems. Notice how they relate to the abstract principles discussed earlier. Heuristics for other products, adapted from Nielsen's original heuristics, have continued to be developed and include heuristics for evaluating shared groupware (Baker et al, 2002), video games (Pinelle et al, 2008), multiplayer games (Pinelle et al, 2009), online

communities (Preece and Shneiderman, 2009), and information visualization (Forsell and Johansson, 2010). Shneiderman's eight golden rules, a set of guidelines first developed in the mid-1980s, are also frequently adapted for use with different types of systems and environments and used as heuristics for identifying usability problems (Shneiderman and Plaisant, 2010; <http://www.cs.umd.edu/~ben/goldenrules.html>):

1. **Strive for consistency.** Consistent sequences of actions should be required in similar situations: identical terminology should be used in prompts, menus, and help screens; and consistent color, layout, capitalization, fonts, and so on should be employed throughout. Exceptions, such as required confirmation of the delete command or no echoing of passwords, should be comprehensible and limited in number.
2. **Cater to universal usability.** Recognize the needs of diverse users and design for plasticity, facilitating transformation of content. Novice to expert differences, age ranges, disabilities, and technological diversity each enrich the spectrum of requirements that guides design. Adding features for novices, such as explanations, and features for experts, such as shortcuts and faster pacing, can enrich the interface design and improve perceived system quality.
3. **Offer informative feedback.** For every user action, there should be system feedback. For frequent and minor actions, the response can be modest, whereas for infrequent and major actions, the response should be more substantial. Visual presentation of the objects of interest provides a convenient environment for showing changes explicitly.
4. **Design dialogs to yield closure.** Sequences of actions should be organized into groups with a beginning, middle, and end. Informative feedback at the completion of a group of actions gives operators the satisfaction of accomplishment, a sense of relief, a signal to drop contingency plans from their minds, and an indicator to prepare for the next group of actions. For example, e-commerce websites move users from selecting products to the checkout, ending with a clear confirmation page that completes the transaction.
5. **Prevent errors.** As much as possible, design the system such that users cannot make serious errors; for example, gray out menu items that are not appropriate and do not allow alphabetic characters in numeric entry fields. If a user makes an error, the interface should detect the error and offer simple, constructive, and specific instructions for recovery. For example, users should not have to retype an entire name-address form if they enter an invalid zip code, but rather should be guided to repair only the faulty part. Erroneous actions should leave the system state unchanged, or the interface should give instructions about restoring the state.
6. **Permit easy reversal of actions.** As much as possible, actions should be reversible. This feature relieves anxiety, since the user knows that errors can be undone, and encourages exploration of unfamiliar options. The units of reversibility may be a single action, a data-entry task, or a complete group of actions, such as entry of a name-address block.

7. Support internal locus of control. Experienced users strongly desire the sense that they are in charge of the interface and that the interface responds to their actions. They don't want surprises or changes in familiar behavior, and they are annoyed by tedious data-entry sequences, difficulty in obtaining necessary information, and inability to produce their desired result.

8. Reduce short-term memory load. Humans' limited capacity for information processing in short-term memory (the rule of thumb is that we can remember 'seven plus or minus two chunks' of information) requires that designers avoid interfaces in which users must remember information from one screen and then use that information on another screen. It means that smartphones should not require re-entry of phone numbers, website locations should remain visible, multiple-page displays should be consolidated, and sufficient training time should be allotted for complex sequences of actions.

1. **Strive for consistency** in action sequences, layout, terminology, command use and so on.
2. **Enable frequent users to use shortcuts**, such as abbreviations, special key sequences and macros, to perform regular, familiar actions more quickly.
3. **Offer informative feedback** for every user action, at a level appropriate to the magnitude of the action.
4. **Design dialogs to yield closure** so that the user knows when they have completed a task.
5. **Offer error prevention** and simple error handling so that, ideally, users are prevented from making mistakes and, if they do, they are offered clear and informative instructions to enable them to recover.
6. **Permit easy reversal of actions** in order to relieve anxiety and encourage exploration, since the user knows that he can always return to the previous state.
7. **Support internal locus of control** so that the user is in control of the system, which responds to his actions.
8. **Reduce short-term memory load** by keeping displays simple, consolidating multiple page displays and providing time for learning action sequences.

These rules provide a useful shorthand for the more detailed sets of principles described earlier. Like those principles, they are not applicable to every eventuality and need to be interpreted for each new situation. However, they are broadly useful and their application will only help most design projects.

Doing Heuristic Evaluation

Heuristic evaluation has three stages:

1. The briefing session, in which the experts are told what to do. A prepared script is useful as a guide and to ensure each person receives the same briefing.

2. The evaluation period, in which each expert typically spends 1–2 hours independently inspecting the product, using the heuristics for guidance. The experts need to take at least two passes through the interface. The first pass gives a feel for the flow of the interaction and the product's scope. The second pass allows the evaluator to focus on specific interface elements in the context of the whole product, and to identify potential usability problems. If the evaluation is for a functioning product, the evaluators need to have some specific user tasks in mind so that exploration is focused. Suggesting tasks may be helpful but many experts suggest their own tasks. However, this approach is less easy if the evaluation is done early in design when there are only screen mockups or a specification; the approach needs to be adapted to the evaluation circumstances. While working through the interface, specification, or mockups, a second person may record the problems identified, or the evaluator may think aloud. Alternatively, she may take notes herself. Evaluators should be encouraged to be as specific as possible and to record each problem clearly.

3. The debriefing session, in which the evaluators come together to discuss their findings and to prioritize the problems they found and suggest solutions.

The heuristics focus the evaluators' attention on particular issues, so selecting appropriate heuristics is critically important. Even so, there is sometimes less agreement among evaluators than is desirable, as discussed in the next Dilemma.

There are fewer practical and ethical issues in heuristic evaluation than for other methods because users are not involved. A week is often cited as the time needed to train evaluators (Nielsen and Mack, 1994), but this depends on the person's initial expertise. Typical users can be taught to do heuristic evaluation, although there have been claims that this approach is not very successful (Nielsen, 1994a). A variation of this method is to take a team approach that may also involve users.

Walkthroughs

Walkthroughs have been used by system developers for many years for inspecting code and the concept offers an alternative approach to heuristic evaluation for predicting users' problems without doing user testing. As the name suggests, walkthroughs involve walking through a task with the product and noting problematic usability features. Most walkthrough methods do not involve users. Others, such as pluralistic walkthroughs, involve a team that may include users, as well as developers, and usability specialists. In this section we consider cognitive and pluralistic walkthroughs. Both were originally developed for desktop systems but, as with heuristic evaluation, they can be adapted to web-based systems, smartphones and mobile devices, tabletops, and products such as DVD players.

Cognitive Walkthroughs

"Cognitive walkthroughs involve simulating a user's problem-solving process at each step in the human-computer dialog, checking to see if the user's goals and memory for actions can be

assumed to lead to the next correct action” (Nielsen and Mack, 1994, p. 6). The defining feature is that they focus on evaluating designs for ease of learning – a focus that is motivated by observations that users learn by exploration (Wharton et al, 1994). The steps involved in cognitive walkthroughs are:

1. The characteristics of typical users are identified and documented and sample tasks are developed that focus on the aspects of the design to be evaluated. A description, mockup, or prototype of the interface to be developed is also produced, along with a clear sequence of the actions needed for the users to complete the task.
2. A designer and one or more expert evaluators come together to do the analysis.
3. The evaluators walk through the action sequences for each task, placing it within the context of a typical scenario, and as they do this they try to answer the following questions:
 - Will the correct action be sufficiently evident to the user? (Will the user know what to do to achieve the task?)
 - Will the user notice that the correct action is available? (Can users see the button or menu item that they should use for the next action? Is it apparent when it is needed?)
 - Will the user associate and interpret the response from the action correctly? (Will users know from the feedback that they have made a correct or incorrect choice of action?)

In other words: will users know what to do, see how to do it, and understand from feedback whether the action was correct or not?

4. As the walkthrough is being done, a record of critical information is compiled in which:
 - The assumptions about what would cause problems and why are identified.
 - Notes about side issues and design changes are made.
 - A summary of the results is compiled.
5. The design is then revised to fix the problems presented. As with heuristic and other evaluation methods, developers and researchers can modify the method to meet their own needs more closely, or adapt the process so that it applies to particular features of mobile or other new technologies.

When doing a cognitive walkthrough it is important to document the process, keeping account of what works and what doesn't. A standardized feedback form can be used in which answers are recorded to each question. Any negative answers are carefully documented on a separate form, along with details of the system, its version number, the date of the evaluation, and the evaluators' names. It is also useful to document the severity of the problems: for example, how likely a problem is to occur and how serious it will be for users. The form can also record the process details outlined in points 1 to 4 as well as the date of the evaluation.

Compared with heuristic evaluation, this technique focuses more closely on identifying specific user problems at a high level of detail. Hence, it has a narrow focus that is useful for focusing on

certain types of system rather than for evaluating whole systems. In particular, it can be useful for applications involving complex cognitive operations. However, it is very time consuming and laborious to do and evaluators need a good understanding of the cognitive processes involved.

Pluralistic Walkthroughs

“Pluralistic walkthroughs are another type of walkthrough in which users, developers and usability experts work together to step through a [task] scenario, discussing usability issues associated with dialog elements involved in the scenario steps” (Nielsen and Mack, 1994, p. 5). In a pluralistic walkthrough, each of the evaluators is asked to assume the role of a typical user. Scenarios of use, consisting of a few prototype screens, are given to each evaluator who writes down the sequence of actions they would take to move from one screen to another, without conferring with fellow panelists. Then the panelists discuss the actions they each suggested before moving on to the next round of screens. This process continues until all the scenarios have been evaluated (Bias, 1994).

The benefits of pluralistic walkthroughs include a strong focus on users' tasks at a detailed level, i.e. looking at the steps taken. This level of analysis can be invaluable for certain kinds of systems, such as safety-critical ones, where a usability problem identified for a single step could be critical to its safety or efficiency. The approach lends itself well to participatory design practices by involving a multidisciplinary team in which users play a key role. Furthermore, the group brings a variety of expertise and opinions for interpreting each stage of an interaction. Limitations include having to get all the experts together at once and then proceed at the rate of the slowest. Furthermore, only a limited number of scenarios, and hence paths through the interface, can usually be explored because of time constraints.

Analytics

Analytics is a method for evaluating user traffic through a system – mostly such systems involve selling a product or service so knowing what customers do and want is important for improving the product or service, but analytics are also being used for evaluating non-transactional systems (e.g. Sleeper et al, 2014). When used to examine traffic on a website or part of a website, the analytics are known as web analytics. Web analytics can be collected locally or remotely across the Internet by logging user activity, counting and analyzing the data in order to understand what parts of the website are being used and when. Although analytics are a form of evaluation that is particularly useful for evaluating the usability of a website, they are also valuable for business planning including the business of online education. Recently, analytics have been applied to understand how learners in massive open online courses (MOOCs) and other web-based education courses interact with these systems (e.g. Oviatt et al, 2013). For example, the developers of these systems are interested in such questions as: Why do learners enroll in these systems and at what point do learners tend to drop out and why? What are the characteristics of learners who complete the course compared with those who do not complete it?

Many companies are developing their own analytics tools; others use the services of companies such as Google and VisiStat, which specialize in providing analytics and the analysis necessary to understand large volumes of data. Typically these companies present their analysis in forms that are easy for developers and website managers to understand – e.g. as graphs, tables, and other types of data visualizations. The home page of VisiStat (VisiStat.com) contains a short promotional video describing the kinds of services that the company offers and why these services are important to businesses. An example of how web analytics can be used to analyze and help developers to improve website performance is provided by VisiStat's analysis of Mountain Wines' website (VisiStat, 2010).

Mountain Wines, located in Saratoga, California, aims to create memorable experiences for guests who visit the vineyard. Following on the tradition started by Paul Masson, world famous wine maker, Mountain Wines offers a beautiful venue for a variety of events including weddings, corporate meetings, dinner parties, birthdays, concerts, and vacations. Mountain Wines uses a variety of advertising media to attract customers to its website and in 2010 invested about \$10 000 a month in advertising. However, the website has remained unchanged for several years because the company didn't have a way of evaluating its effectiveness and could not decide whether to increase or decrease investment in it. They then decided to employ VisiStat, to use their web analytics tool. Prior to enlisting this company, the only record that Mountain Wines had of the effectiveness of its advertising came from its front-desk employees who were instructed to ask visitors 'How did you hear about Mountain Wines?'

VisiStat provided Mountain Wines with data showing how their website was being used by potential customers, e.g. data like that shown in Figures 15.4 to 15.6. Figure 15.4 provides an overview of the number of page views of the website per day. Figure 15.5 provides additional details and shows the hour-by-hour traffic. Clicking on the first icon for more detail shows where the IP addresses of the traffic are located (Figure 15.6). VisiStat can also provide information about such things as which visitors are new to the site, which are returners, and which other pages visitors came from.



Figure 15.4 A general view of the kind of data provided by VisiStat

Source: <http://www.visistat.com/tracking/monthly-page-views.php>

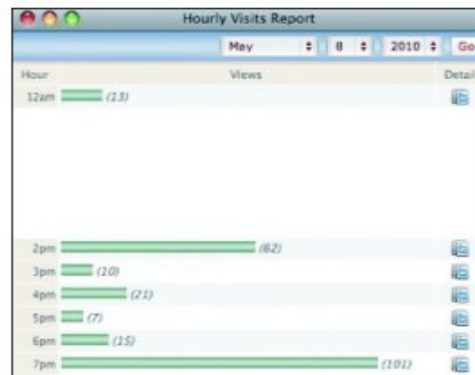


Figure 15.5 Clicking on May 8 provides an hourly report from midnight until 10.00 p.m. (only midnight and 2.00 p.m.–7.00 p.m. shown)

Source: <http://www.visistat.com/tracking/monthly-page-views.php>

	Unique Visitor	Views	Detail
1.	Los Angeles, California	6	
2.	Sharpsburg, Maryland	1	
3.	Phoenix, Arizona	3	
4.	Lemesos, Limassol	2	
5.	Targu-mures, Mures	1	

Figure 15.6 Clicking on the icon for the first hour in Figure 15.5 shows where the IP addresses of the 13 visitors to the website are located

Using this data and other data provided by VisiStat, Mountain Wines could see visitor totals, traffic averages, traffic sources, visitor activity, and more. They discovered the importance of visibility for their top search words; they could pinpoint where guests were going on their website; and they could see where their guests were geographically located.

More recently, other types of specialist analytics have also been developed such as visual analytics, in which thousands and sometimes millions of data points are displayed and manipulated visually, such as Hansen et al's (2011) social network analysis (see Figure 15.7).

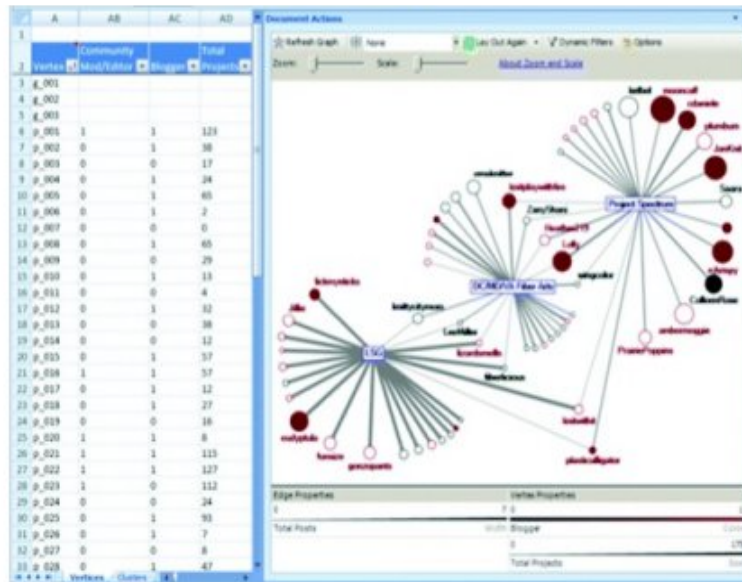


Figure 15.7 Social network analysis showing clusters and the relationships between the entities and the clusters

Source: Perer, A. and Shneiderman, B. (2008) Integrating Statistics and Visualization: Case Studies of Gaining Clarity during Exploratory Data Analysis. CHI 2008 Proceedings. Visual Synthesis April 5–10, 2008, 265–274. ©2008 Association for Computing Machinery, Inc. Reprinted by permission.

Lifelogging is another interesting variation that can be used for evaluation as well as for sharing information with friends, family, and colleagues. Typically, lifelogging involves recording GPS location data and personal interaction data on smartphones. In an evaluation context this can raise privacy concerns. Even though users tend to get used to being logged, they generally want to remain in control of the logging (Kärkkäinen et al, 2010).